

I'll provide a comprehensive explanation of linked lists and their implementations in Java, approaching this topic systematically to build a strong foundation.

Let's start with understanding linked lists fundamentally.

LECTURE PART

1. Introduction to Linked Lists

A linked list is a dynamic data structure where elements (nodes) are stored in non-contiguous memory locations, connected through references or links. Unlike arrays, linked lists can grow or shrink during runtime without requiring reallocation of the entire structure.

Basic Node Structure in Java:

```
```java
class Node<T> {
 T data; // The actual data stored in the node
 Node<T> next; // Reference to the next node

 public Node(T data) {
 this.data = data;
 this.next = null;
 }
}
```
```

2. Types of Linked Lists

A. Singly Linked Lists

This is the simplest form of a linked list where each node points to the next node in the sequence.

Implementation example:

```
```java
class SinglyLinkedList<T> {
 private Node<T> head;
 private int size;

 public void add(T data) {
 Node<T> newNode = new Node<>(data);
 if (head == null) {
 head = newNode;
 } else {
 Node<T> current = head;
 while (current.next != null) {
 current = current.next;
 }
 current.next = newNode;
 }
 size++;
 }

 public void remove(T data) {

```

```

 if (head == null) return;

 if (head.data.equals(data)) {
 head = head.next;
 size--;
 return;
 }

 Node<T> current = head;
 while (current.next != null && !current.next.data.equals(data)) {
 current = current.next;
 }

 if (current.next != null) {
 current.next = current.next.next;
 size--;
 }
}
}
...

```

## B. Doubly Linked Lists

Each node contains references to both the next and previous nodes, allowing bidirectional traversal.

```

```java
class DoublyNode<T> {
    T data;
    DoublyNode<T> next;
    DoublyNode<T> prev;

    public DoublyNode(T data) {
        this.data = data;
        this.next = null;
        this.prev = null;
    }
}

class DoublyLinkedList<T> {
    private DoublyNode<T> head;
    private DoublyNode<T> tail;

    public void add(T data) {
        DoublyNode<T> newNode = new DoublyNode<>(data);
        if (head == null) {
            head = tail = newNode;
        } else {
            tail.next = newNode;
            newNode.prev = tail;
            tail = newNode;
        }
    }
}

```

...

C. Circular Linked Lists

The last node points back to the first node, creating a circle. This can be implemented with either singly or doubly linked lists.

```
```java
class CircularLinkedList<T> {
 private Node<T> head;
 private Node<T> tail;

 public void add(T data) {
 Node<T> newNode = new Node<>(data);
 if (head == null) {
 head = tail = newNode;
 tail.next = head; // Creating the circular reference
 } else {
 tail.next = newNode;
 tail = newNode;
 tail.next = head; // Maintaining the circular reference
 }
 }
}
```
```

D. Node-based Storage with Arrays

This is a hybrid approach where we use an array to store references to nodes, combining the benefits of both arrays and linked lists.

```
```java
class NodeArray<T> {
 private Node<T>[] nodeArray;
 private int capacity;

 @SuppressWarnings("unchecked")
 public NodeArray(int capacity) {
 this.capacity = capacity;
 this.nodeArray = (Node<T>[]) new Node[capacity];
 }

 public void set(int index, T data) {
 if (index >= 0 && index < capacity) {
 nodeArray[index] = new Node<>(data);
 }
 }
}
```
```

LAB IMPLEMENTATIONS

1. Circular Queue using Linked List:

```
```java
```

```

class CircularQueue<T> {
 private Node<T> rear;
 private int size;
 private int capacity;

 public CircularQueue(int capacity) {
 this.capacity = capacity;
 this.rear = null;
 this.size = 0;
 }

 public boolean enqueue(T data) {
 if (isFull()) return false;

 Node<T> newNode = new Node<>(data);
 if (isEmpty()) {
 newNode.next = newNode; // Point to itself
 rear = newNode;
 } else {
 newNode.next = rear.next; // New node points to front
 rear.next = newNode; // Rear points to new node
 rear = newNode; // Update rear
 }
 size++;
 return true;
 }

 public T dequeue() {
 if (isEmpty()) return null;

 T data = rear.next.data; // Get front element
 if (rear.next == rear) { // Only one element
 rear = null;
 } else {
 rear.next = rear.next.next; // Skip the front element
 }
 size--;
 return data;
 }

 public boolean isEmpty() {
 return rear == null;
 }

 public boolean isFull() {
 return size == capacity;
 }
}

```

## 2. Stack using Linked List:

```

```java

```

```

class LinkedStack<T> {
    private Node<T> top;
    private int size;

    public void push(T data) {
        Node<T> newNode = new Node<>(data);
        newNode.next = top;
        top = newNode;
        size++;
    }

    public T pop() {
        if (isEmpty()) return null;

        T data = top.data;
        top = top.next;
        size--;
        return data;
    }

    public T peek() {
        return isEmpty() ? null : top.data;
    }

    public boolean isEmpty() {
        return top == null;
    }

    public int size() {
        return size;
    }
}

```

Key Points to Remember:

1. Linked lists provide dynamic memory allocation and efficient insertion/deletion operations.
2. Singly linked lists are memory-efficient but only allow forward traversal.
3. Doubly linked lists consume more memory but enable bidirectional traversal.
4. Circular linked lists are useful for applications requiring cyclic data structures.
5. The choice between different types depends on specific use cases and requirements.

These implementations demonstrate the fundamental concepts of linked lists and their practical applications in creating other data structures. Each implementation includes proper encapsulation, error handling, and basic operations necessary for their respective functionalities.

Would you like me to elaborate on any specific aspect or provide more examples of operations on these data structures?